SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

Duration: 1 Hour

QUESTIONS: 3 **Total Marks:30**

Annexure A

sDry Run Evaluation

Code of Conduct:

I ***JEROLIN MARY R (192411024)*** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph:

```
        A
       / \
      B   C
     /\   /\
    D  E F  G
```

Task: Starting from node **A**, perform a **Breadth-First Search (BFS)**. Complete the following table.

Iteration	Queue	Visited Node
1		
2		
3		
4		
5		
6		
7		

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

2. Depth-First Search (DFS) Using the same graph,

```
      A
     / \
    B   C
   /\   /\
  D  E F  G
```

Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

Perform **Depth-First Search (DFS)** starting from **A**.

Complete the table.

3. **Initial State :**

   1 3 6
   5 0 2
   4 7 8

Goal State :

1 2 3
4 5 6
7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.
Fill in the table.

Iteration	**Current State**	**Queue Before Expansion**	**Queue After Expansion**
1			
2			
3			
4			
5			

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.

ANSWERS:

1. Breadth-First Search (BFS)

Given Graph:

A → B, C
B → D, E
C → F, G

BFS Table

Iteration | Queue | Visited Node
1 | [B, C] | A
2 | [C, D, E] | B
3 | [D, E, F, G] | C
4 | [E, F, G] | D
5 | [F, G] | E
6 | [G] | F
7 | [] | G

Answers

1. BFS Traversal Order:

A → B → C → D → E → F → G

2. Queue contents after each iteration:

Iteration 1: [B, C]
Iteration 2: [C, D, E]
Iteration 3: [D, E, F, G]
Iteration 4: [E, F, G]
Iteration 5: [F, G]
Iteration 6: [G]
Iteration 7: []

2. Depth-First Search (DFS)

Given Graph:

A → B, C
B → D, E
C → F, G

Assuming the left child is visited first.

DFS Table

Iteration | Stack | Visited Node
1 | [C, B] | A
2 | [C, E, D] | B
3 | [C, E] | D
4 | [C] | E
5 | [G, F] | C
6 | [G] | F
7 | [] | G

DFS Traversal Order

A → B → D → E → C → F → G

3. 8-Puzzle BFS

Initial State:

1 3 6
5 0 2
4 7 8

Goal State:

1 2 3
4 5 6
7 8 0

Move order used: Up → Down → Left → Right.

State Names

S0 = Initial State

1 3 6
5 0 2
4 7 8

S1 = Move Up

1 0 6
5 3 2
4 7 8

S2 = Move Down

1 3 6
5 7 2
4 0 8

S3 = Move Left

1 3 6
0 5 2
4 7 8

S4 = Move Right

1 3 6
5 2 0
4 7 8

BFS Table

Iteration | Current State | Queue Before Expansion | Queue After Expansion

1 | S0 | [S0] | [S1, S2, S3, S4]

2 | S1 | [S1, S2, S3, S4] | [S2, S3, S4, S5, S6]

3 | S2 | [S2, S3, S4, S5, S6] | [S3, S4, S5, S6, S7, S8]

4 | S3 | [S3, S4, S5, S6, S7, S8] | [S4, S5, S6, S7, S8, S9, S10]

5 | S4 | [S4, S5, S6, S7, S8, S9, S10] | [S5, S6, S7, S8, S9, S10, S11, S12]

Questions

1. Which state is expanded in each iteration?

Iteration 1: S0
Iteration 2: S1
Iteration 3: S2
Iteration 4: S3
Iteration 5: S4

BFS always expands the state at the front of the queue.

2. How many states are generated in total?

311 unique states are generated before the goal is first reached/discovered.

3. Complete Solution Path

Initial State:

1 3 6
5 0 2
4 7 8

Move 1: Right

1 3 6
5 2 0
4 7 8

Move 2: Up

1 3 0
5 2 6
4 7 8

Move 3: Left

1 0 3
5 2 6
4 7 8

Move 4: Down

1 2 3
5 0 6
4 7 8

Move 5: Left

1 2 3
0 5 6
4 7 8

Move 6: Down

1 2 3
4 5 6
0 7 8

Move 7: Right

1 2 3
4 5 6
7 0 8

Move 8: Right

1 2 3
4 5 6
7 8 0

Solution:

Right → Up → Left → Down → Left → Down → Right → Right

Number of moves = 8

4. Why does BFS always find the shortest solution?

BFS explores the states level by level. It first explores all states requiring 1 move, then 2 moves, then 3 moves, and so on. Therefore, when BFS reaches the goal for the first time, the solution has the minimum number of moves.